D. Y. Patil College of Engineering and Technology, Kolhapur

Department of Computer Science & Engineering

Class: Final Year B. Tech. Div.: A

Subject: Non Structured Database

Experiment no: 10

Group No. 3 Mini Project

Title of Mini-Project:

CineGraph — Movie Recommendation System Using Neo4j Graph Database

"Discover Movies Through Connections"

Problem Statement:

People often spend a long time deciding which movie to watch. Most apps show the same "top rated" list to everyone and never explain why a movie is suggested. Movie data is full of connections: a user likes a movie, the movie has a genre, actors and a director, and other users like the same movies. In a normal table (SQL) database, following these connections needs many JOIN operations, and the queries become long and slow.

Aim: To build a movie recommendation website, **CineGraph**, that stores users, movies, genres, actors and directors as a **graph in Neo4j**, suggests movies by following these connections with **Cypher** queries, and shows the reason for every suggestion.

Objectives

- To store movie data and user activity as a graph in the Neo4j database.
- To recommend movies using genres, actors, directors and users with similar taste.
- To show a clear reason and a match score for every recommendation.
- To give useful suggestions even to new users who have no history (cold start).
- To show the graph visually so that the connections are easy to understand.

Introduction:

What is a non-structured (NoSQL) database?

A **non-structured (NoSQL) database** does not store data in fixed tables with rows and columns. There are four common types, shown in Table 1. Each type is good for a different kind of data.

Table 1: *Types of NoSQL databases*

Type	How data is stored	Example
Key–value	A key and its value	Redis
Document	JSON-like documents	MongoDB
Column-family	Rows with flexible columns	Cassandra
Graph	Nodes and relationships	**Neo4j** (used in this project)

A **graph database** stores data as **nodes** (things, such as a movie) and **relationships** (links between things, such as “user LIKED movie”). A recommendation is really about finding connected items, so a graph database is a natural choice.

About CineGraph

CineGraph is a full web application. Its main features are:

- Browse and search 80 movies, with filters for genre, year and rating.
- Sign up, log in, like movies, give 1–5 star ratings and keep a watch history.
- Personal recommendations with a match percentage and a clear reason (for example, same director or same actors).
- Four recommendation methods: genre-based, actor/director-based, similar users and hybrid.
- An interactive graph view that shows how movies, genres, actors, directors and users are connected.
- An insights page with statistics and charts calculated directly from the graph.

Table 2: *Technologies used*

Part	Technology
Frontend (website)	React, Vite, Tailwind CSS, React Router, Axios
Graph view and charts	Cytoscape.js (graph explorer), Recharts (charts)
Backend (server)	Node.js with Express (REST API)
Login and security	JWT in an httpOnly cookie, bcrypt password hashing, input validation
Database	Neo4j graph database with the Cypher query language

Part	Technology
Testing and tools	Vitest, Supertest, VS Code, Git, Neo4j Browser

System Architecture:

CineGraph uses a simple **three-tier architecture**, shown in Figure 1. Each tier has its own job, which keeps the system easy to understand and maintain.

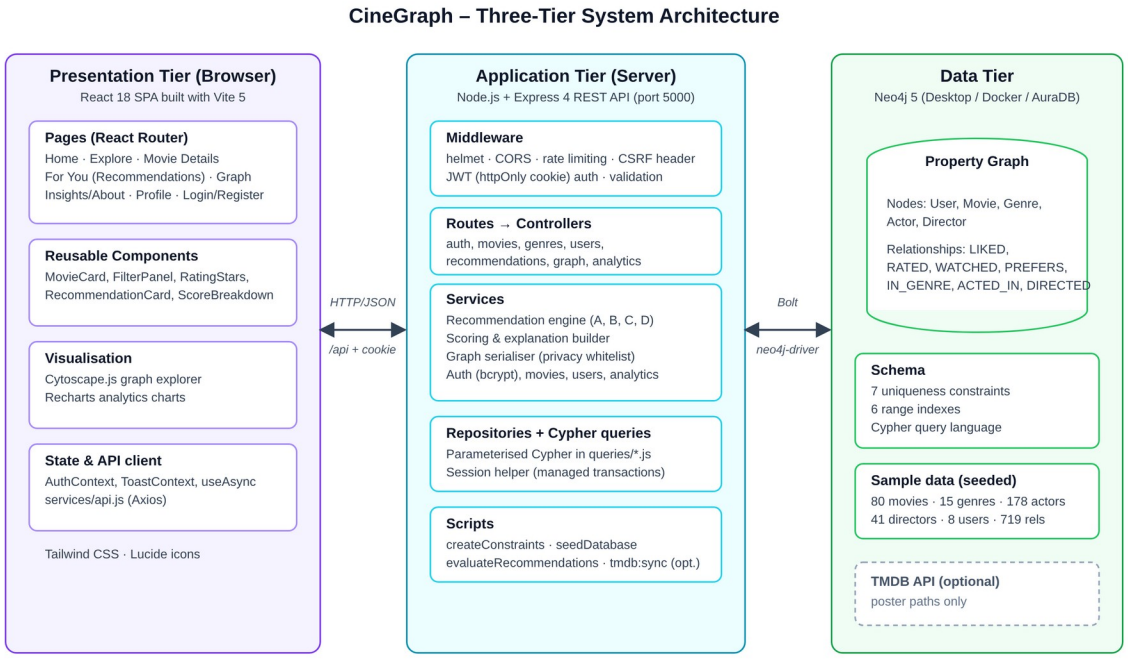


Figure 1: *System architecture of CineGraph*

- **Presentation tier (Browser):** the React website that the user sees. It has pages for Home, Explore, Movie details, Recommendations (“For You”), Graph, Insights, Profile and Login. It sends requests to the server and shows the results.
- **Application tier (Server):** a Node.js + Express REST API. It checks the login, validates the input, runs the recommendation logic, builds the explanations and prepares the graph data for the graph view.
- **Data tier (Database):** Neo4j stores all nodes and relationships. The server talks to Neo4j using the official Neo4j JavaScript driver and Cypher queries over the Bolt protocol.

Flow of data: the user clicks something on the website → the website sends an HTTP request to the API → the API runs a Cypher query on Neo4j → Neo4j returns the matching nodes → the API sends the result back as JSON → the website displays it.

Security

- Passwords are never stored directly; only bcrypt hashes are saved.
- After login, the server gives the browser a secure cookie (JWT). Every request uses it to know who the user is, so one user cannot change another user's data.
- All inputs are checked, and all Cypher queries use parameters, which prevents injection attacks.

Module description or working of system:

Modules

1. **User module:** registration, login and logout. The profile page shows liked, rated and watched movies, favourite genres and preferred genres.
2. **Movie catalogue module:** lists the movies with search, filters (genre, year, rating), sorting and "load more" pages. The filtering is done by Cypher inside Neo4j.
3. **Movie details module:** shows the story, cast, director and ratings. The user can like, rate (1–5 stars) or mark the movie as watched, and see related movies with the reason they are related.
4. **Recommendation module:** finds new movies for the user using four methods (explained below) and shows the reason and score for each one.
5. **Graph explorer module:** draws the movie graph with Cytoscape.js. Each type of node has its own colour and shape. Clicking a node highlights its links; double-clicking loads more connections from Neo4j.
6. **Insights module:** shows counts of movies, users, actors and ratings, and charts such as the most popular genres and the rating distribution.

How the recommendations work

- **Genre-based:** find other movies in the same genres as the movies the user liked.
- **Actor/Director-based:** find other movies with the same actors or the same director.
- **Similar users (collaborative filtering):** find users who liked the same movies, then suggest what they also liked. Similarity is measured with the Jaccard index (shared movies ÷ all movies of both users).
- **Hybrid:** combine all the scores into one match score using the weights in Table 3.

Table 3: *Weights used for the hybrid match score*

Signal	Weight	What it measures
Genre	30%	How well the movie matches the genres the user likes
Actor	15%	Shared actors with movies the user liked
Director	15%	Same director as movies the user liked
Similar users	25%	How many users with similar taste liked the movie
Movie rating	15%	General quality of the movie (rating out of 10)

Movies the user has already liked, rated or watched are never suggested again. Table 4 shows how CineGraph handles users with little or no history.

Table 4: *Handling new users (cold start)*

User has…	What CineGraph shows
No likes, ratings or favourite genres	Popular movies, clearly marked “not personalised”, and asks the user to pick genres
Only favourite genres	Movies from those genres, sorted by rating
At least one like or good rating	Fully personal recommendations using all signals

Working of the system (step by step)

Figure 2 shows how one recommendation request moves through the system.

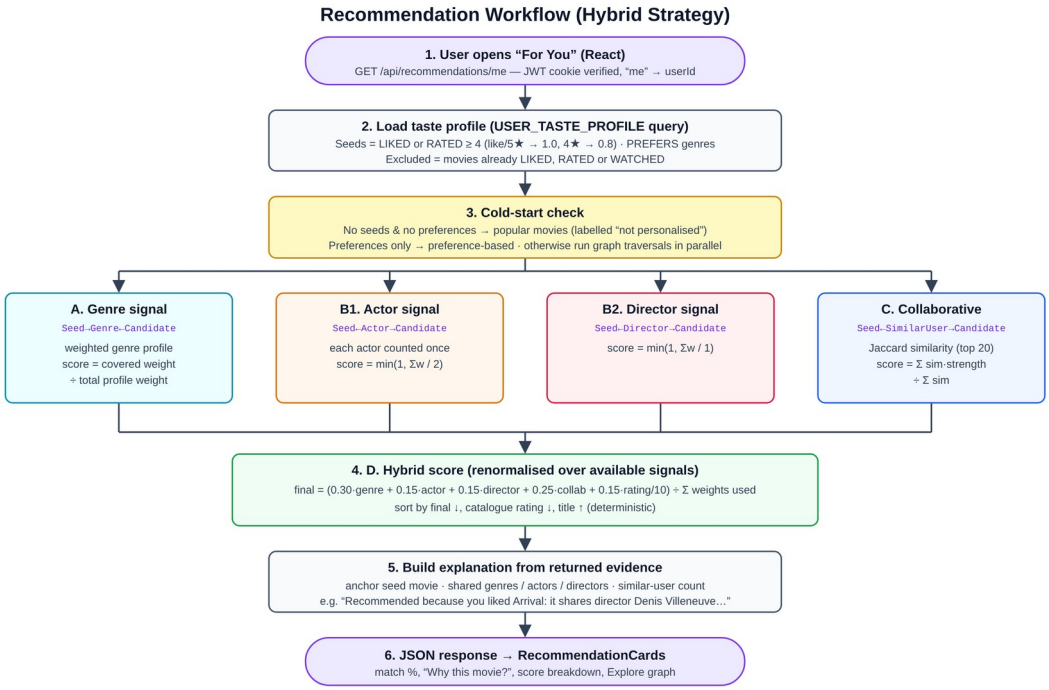


Figure 2: *Recommendation workflow*

1. The user logs in to CineGraph.

2. The user likes or rates movies. Each action is saved in Neo4j as a relationship, for example (User)-[:LIKED]->(Movie).
3. The user opens the “For You” page and the website asks the server for recommendations.
4. The server reads the user’s liked movies, ratings and favourite genres from the graph.
5. The server runs Cypher queries that follow the links to genres, actors, directors and similar users.
6. Each movie gets a score; the list is sorted and a reason is written, such as “same director Christopher Nolan”.
7. The website shows the movies with their match percentage and reason. Any new like or rating changes the next results.

Detail description of Nonstructured database used in miniproject:

Introduction to Neo4j

Neo4j is one of the most widely used graph databases. It stores data as a **property graph**:

- **Node** – an object such as a User or a Movie. Its type is called a **label**.
- **Relationship** – a named, directed link between two nodes, such as LIKED or ACTED_IN.
- **Property** – a value stored on a node or relationship, such as title or rating.

Neo4j is queried with **Cypher**, a language that looks like a drawing of the graph: (u:User)-[:LIKED]->(m:Movie). Neo4j keeps every relationship directly connected to its nodes, so it can jump from one node to the next without searching whole tables. CineGraph was tested with Neo4j 5.26 and also works with Neo4j Desktop and Neo4j AuraDB (cloud).

Why Neo4j was chosen

Table 5: *SQL database compared with Neo4j for this project*

Point	SQL database	Neo4j
Storing links	Extra tables and foreign keys	Relationships are stored directly
“Movies by the same director as a movie I liked”	Many JOINs over several tables	One short Cypher pattern
Adding new kinds of data	Table structure must be changed	New labels and relationships can be added any time
Showing the data as a graph	Needs extra work	Results are already a graph

Graph design of CineGraph

Figure 3 shows the graph design. There are five types of nodes and seven types of relationships.

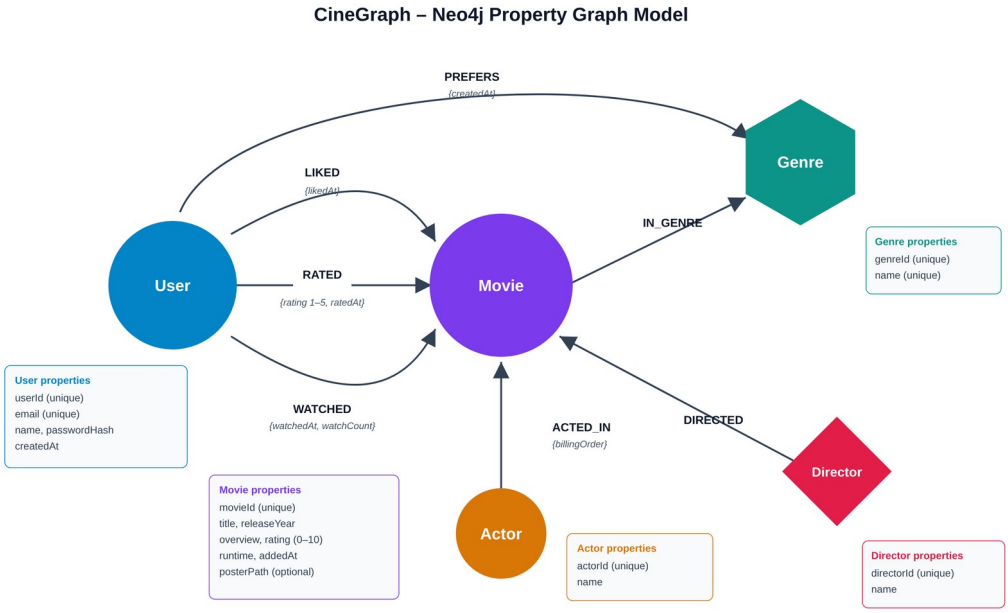


Figure 3: *Graph model (nodes, relationships and properties) of CineGraph*

Table 6: *Data stored in the sample database*

Item	Count	Item	Count
Movie nodes	80	IN_GENRE links	220
Actor nodes	178	ACTED_IN links	268
Director nodes	41	DIRECTED links	87
Genre nodes	15	LIKED / RATED / WATCHED links	38 / 26 / 62
User nodes	8	PREFERS links	18

Cypher queries used in the project

1. Constraint: a uniqueness constraint stops duplicate nodes, for example two “Inception” movies.

```
CREATE CONSTRAINT movie_id_unique IF NOT EXISTS
FOR (m:Movie) REQUIRE m.movieId IS UNIQUE;
```

2. Saving a like: `MERGE` creates the LIKED relationship only if it does not already exist, so a movie cannot be liked twice.

```
MATCH (u:User {userId: $userId}), (m:Movie {movieId: $movieId})
MERGE (u)-[r:LIKED]->(m)
ON CREATE SET r.likedAt = datetime()
RETURN m.title AS liked
```

3. Saving a rating: the same idea; if the user rates the movie again, the old rating is simply updated.

```
MATCH (u:User {userId: $userId}), (m:Movie {movieId: $movieId})
MERGE (u)-[r:RATED]->(m)
SET r.rating = $rating, r.ratedAt = datetime()
RETURN m.title AS movie, r.rating AS rating
```

4. Genre-based recommendation: start from the user, go to the movies they liked, then to their genres, then to other movies in those genres that the user has not seen yet.

```
MATCH (u:User {userId: $userId})-[:LIKED]->(:Movie)-[:IN_GENRE]->(g:Genre)<-
[:IN_GENRE]-(rec:Movie)
WHERE NOT EXISTS { (u)-[:LIKED|RATED|WATCHED]->(rec) }
RETURN rec.title AS movie, rec.rating AS rating, count(DISTINCT g) AS sharedGenres
ORDER BY sharedGenres DESC, rating DESC LIMIT 5
```

5. Actor-based recommendation: find movies that share actors with the movies the user liked.

```
MATCH (u:User {userId: $userId})-[:LIKED]->(:Movie)<-[:ACTED_IN]-(a:Actor)-
[:ACTED_IN]->(rec:Movie)
WHERE NOT EXISTS { (u)-[:LIKED|RATED|WATCHED]->(rec) }
RETURN rec.title AS movie, collect(DISTINCT a.name) AS sharedActors
ORDER BY size(sharedActors) DESC, movie ASC LIMIT 5
```

For the demo user Aarav this returns, for example, “The Dark Knight Rises” with the shared actors Anne Hathaway, Joseph Gordon-Levitt and Tom Hardy.

6. Similar users: find other users who liked the same movies and suggest the movies they also liked.

```
MATCH (u:User {userId: $userId})-[:LIKED]->(:Movie)<-[:LIKED]-(other:User)-[:LIKED]-
>(rec:Movie)
WHERE other <> u AND NOT EXISTS { (u)-[:LIKED|RATED|WATCHED]->(rec) }
RETURN rec.title AS movie, rec.rating AS rating, count(DISTINCT other) AS
similarUsers
ORDER BY similarUsers DESC, rating DESC LIMIT 5
```

7. Statistics for the Insights page: `COUNT { }` counts the nodes or relationships that match a pattern.

```
RETURN COUNT { (:Movie) } AS movies, COUNT { (:User) } AS users,
       COUNT { (:Actor) } AS actors, COUNT { ()-[:RATED]->() } AS ratings
```

Queries 4, 5 and 6 are simplified forms of the project’s queries. The full versions also use ratings, preferred genres and a similarity score, but follow exactly the same idea of walking along the graph. All queries above were run on the project database and work correctly.

Testing and results

- The seed script loads the sample data into Neo4j. Running it a second time creates 0 new nodes, so there are no duplicates.
- 53 unit/API tests and 25 integration tests (run against a real Neo4j database) passed.
- The tests also check that every actor and director named in a recommendation reason really connects the two movies in the graph.

Screenshots:

Screenshots of the running CineGraph application (Neo4j with the sample data):

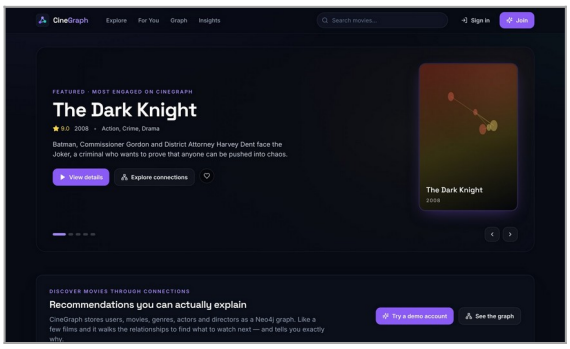


Figure 4: *Home page*

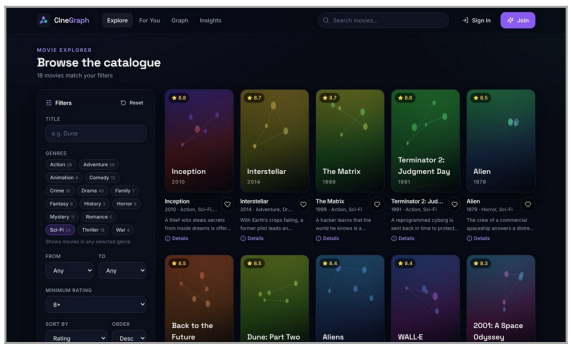


Figure 5: *Movie explorer with genre and rating filters*

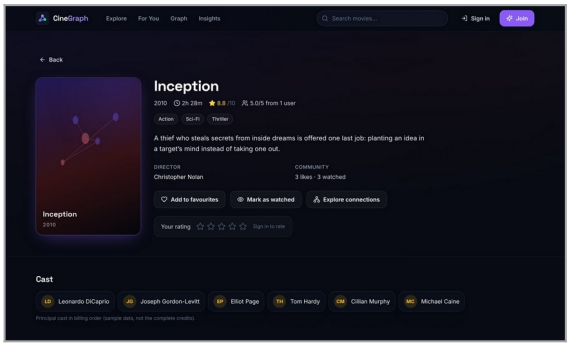


Figure 6: *Movie details with like, watch and rating options*

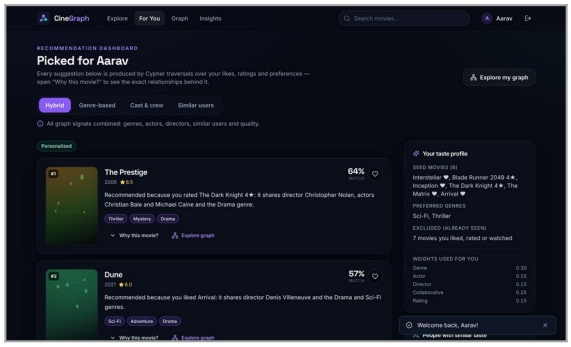


Figure 7: *Personal recommendations (For You page)*

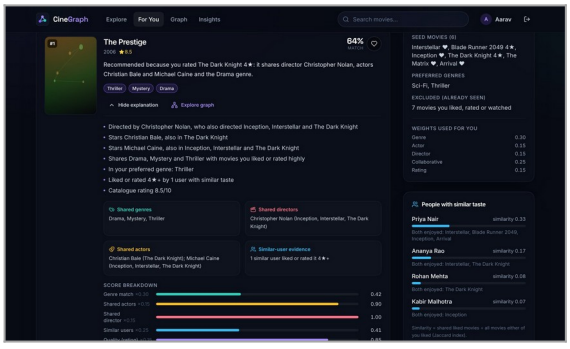


Figure 8: *Reason and score for a recommendation*

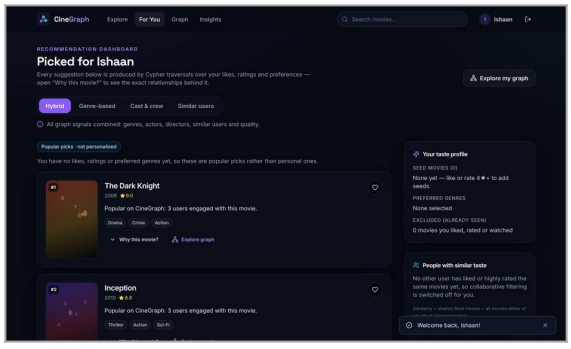


Figure 9: *New user: popular picks (cold start)*

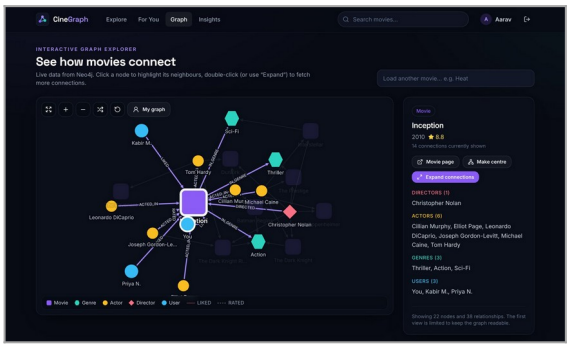


Figure 10: *Interactive graph explorer*

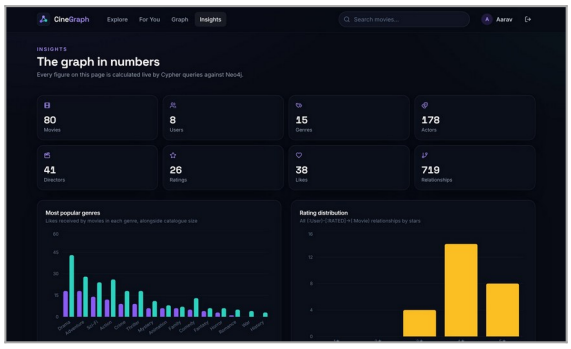


Figure 11: *Insights calculated from the graph*

Group Members:

Unique id	Roll No	Name of Student	Sign
EN23192312	11	Shreeya Amit Kalebere	
EN23120981	12	Shubham Shankar Pattewar	
EN23217963	13	Anuj Arun Powar	
EN23250682	14	Virendra Vilas Katale	
EN23267649	15	Preeti Nishikant Sonule	